

ESI – École nationale Supérieure d'Informatique

Moteur de recherche sémantique et analytique sur des logs massifs Big Data

Rapport technique – TP5

Etudiants	Tati Youcef – <code>my_tati@esi.dz</code> Mostefai Mounir Sofiane – <code>mm_mostefai@esi.dz</code>
Encadrante	Ens. Amrouche Karima – <code>k_amrouche@esi.dz</code>

Année universitaire 2025–2026

Contents

1	Introduction	1
2	Jeu de donnees	1
3	Architecture generale	1
4	Pretraitement Big Data	2
5	Schema de stockage	3
6	Details d'implementation	3
7	Choix de conception	4
8	Vectorisation	4
9	Recherche et analyse	4
9.1	Recherche semantique	5
9.2	Recherche par mots-cles	5
9.3	Logs similaires	5
9.4	Analytique	5
10	Interface de demonstration	5
11	Reproductibilite	5
12	API applicative	6
13	Scenario de demonstration	7
14	Evaluation attendue	7
15	Tests et validation	8
16	Limites et ameliorations	8
17	Conclusion	8

1 Introduction

Les infrastructures modernes produisent des volumes importants de journaux d'exécution. Dans un système distribué, ces journaux proviennent de machines, services, processus et utilisateurs différents. Leur exploitation par recherche textuelle simple reste limitée : une même panne peut être formulée par plusieurs messages proches, et une requête par mot-clé peut ignorer des logs pertinents lorsque les termes exacts ne sont pas présents.

Ce projet réalise une plateforme Big Data capable d'ingérer un grand volume de logs, de les structurer avec Apache Spark, de transformer les messages en vecteurs sémantiques, de stocker ces vecteurs dans PostgreSQL avec pgvector, puis de fournir des fonctions de recherche et d'analyse. L'objectif n'est pas seulement de lancer un modèle d'embedding, mais de construire une chaîne complète, reproductible et démontrable.

Les livrables couverts sont le code source documenté, un pipeline Big Data fonctionnel, une base vectorielle indexée, un rapport technique et une interface web professionnelle. La solution reste entièrement open source et respecte les technologies imposées : Python, Apache Spark, PostgreSQL, pgvector et Sentence-Transformers.

2 Jeu de données

Le dataset choisi est OpenSSH de LogHub-2.0. Il contient 638 947 entrées brutes dans l'extraction locale, ce qui dépasse le seuil de 500 000 entrées textuelles demandé dans le sujet. Le dépôt local contient trois fichiers principaux :

Fichier	Rôle
OpenSSH_full.log	Logs bruts au format syslog, avec mois, jour, heure, hôte, service et message.
OpenSSH_full.log_structured.csv	Version structurée fournie par LogHub : identifiant de ligne, contenu, identifiant d'événement et template.
OpenSSH_full.log_templates.csv	Liste des templates d'événements et leurs occurrences.

Un exemple de ligne brute est :

```
Dec 10 06:55:48 LabSZ sshd[24200]: Failed password for invalid user webmaster
from 173.234.31.186 port 38926 ssh2
```

La version structurée sépare le contenu métier du préfixe syslog. Elle fournit aussi un `EventId` et un `EventTemplate`. Ces templates sont importants car ils réduisent le bruit des valeurs variables comme adresses IP, ports, noms d'utilisateurs et identifiants de processus.

3 Architecture générale

L'architecture suit une chaîne batch classique adaptée à un contexte Big Data :

```
Logs OpenSSH
-> Pretraitement Spark
-> CSV/Parquet traites
-> PostgreSQL + pgvector
-> API FastAPI
-> Interface Streamlit
```

Spark est utilise pour charger les fichiers volumineux, parser les prefixes syslog, joindre le log brut avec le CSV structure, normaliser les messages et produire des sorties persistantes. PostgreSQL stocke les logs nettoyes et une table vectorielle des messages normalises distincts. pgvector ajoute le type `vector(384)` et l'index HNSW pour la recherche approximative de voisins proches. FastAPI expose les fonctionnalites sous forme d'API HTTP, et Streamlit fournit la demonstration interactive.

La separation entre pipeline, base, API et interface rend le projet reproductible : le pipeline peut etre lance en ligne de commande, l'API peut etre testee seule, et l'interface consomme uniquement les endpoints publics.

4 Pretraitement Big Data

La commande `tp5-log-search prepare-data` execute le pretraitement Spark. Le traitement lit le fichier brut, associe un numero de ligne, puis joint ce flux avec le CSV structure par `LineId`. Cette strategie permet de conserver les informations temporelles du log brut et les templates fournis par LogHub.

Les champs extraits sont :

- `line_id`, identifiant stable de la ligne ;
- `log_timestamp`, timestamp reconstruit avec une annee configurable ;
- `host`, `service`, `process_id` ;
- `level`, niveau heuristique : INFO, WARNING, ERROR ou CRITICAL ;
- `raw_message`, message original ;
- `normalized_message`, message normalise vectorise ;
- `event_id` et `event_template`.

Le fichier OpenSSH ne contient pas l'annee. Pour permettre les analyses temporelles, le pipeline utilise une variable `LOG_YEAR`, fixee par default a 2024. Cette hypothese est explicite, configurable et ne modifie pas l'ordre relatif des evenements.

Le resultat est ecrit sous deux formats. Le CSV sert au chargement rapide dans PostgreSQL avec `COPY`. Le Parquet conserve une sortie analytique compacte et partitionnee par niveau de log, utile pour des analyses Spark supplementaires.

5 Schema de stockage

Le schema relationnel est centre sur trois tables :

Table	Description
<code>event_templates</code>	Dictionnaire des templates LogHub et nombre d'occurrences annonce.
<code>log_entries</code>	Lignes de logs nettoyees, metadonnees temporelles, niveau et message.
<code>message_embeddings</code>	Messages normalises distincts, occurrences et vecteurs semantiques indexes.
<code>pipeline_runs</code>	Trace d'execution du pipeline complet.

La colonne vectorielle principale est stockee dans `message_embeddings` :

```
embedding VECTOR(384)
```

La dimension 384 correspond au modele `sentence-transformers/all-MiniLM-L6-v2`. Les index relationnels couvrent le niveau, le temps, l'evenement et la recherche textuelle par trigrammes. La recherche semantique repose sur :

```
CREATE INDEX message_embeddings_embedding_hnsw_idx  
ON message_embeddings USING hnsw (embedding vector_cosine_ops);
```

L'index HNSW est adapte a la recherche de similarite a grande echelle car il evite un parcours sequentiel complet pour chaque requete. La distance utilisee est le cosinus, coherente avec les embeddings normalises.

6 Details d'implementation

Le code source est organise comme un package Python installable. Cette organisation evite les scripts isoles difficiles a maintenir et donne une structure proche d'un projet professionnel. Les modules principaux sont separes par responsabilite : configuration, pretraitement Spark, acces base de donnees, embeddings, recherche, analytique, API, interface et CLI.

Le module de configuration lit les variables d'environnement depuis `.env`. Les chemins sont resolus relativement a la racine du depot afin que les commandes fonctionnent depuis n'importe quel repertoire. Les valeurs par default pointent vers le dataset OpenSSH deja extrait dans `data/raw/OpenSSH`.

Le module Spark evite de supposer que le CSV structure contient toutes les informations utiles. Il reconstruit les metadonnees systeme depuis le log brut, puis joint ces informations avec les templates LogHub. Cette decision conserve a la fois la richesse temporelle du fichier brut et la qualite du parsing fourni par LogHub.

Le chargement PostgreSQL utilise `COPY` plutot que des insertions ligne par ligne. Pour un dataset de plusieurs centaines de milliers de lignes, cette difference est importante : `COPY` reduira le temps de chargement et limite la charge cote client Python. Les commandes peuvent etre relancees car le pipeline initialise le schema puis recharge les tables applicatives de maniere deterministe.

La recherche semantique est encapsulee dans un module dedie. L'API et l'interface n'ont donc pas a connaitre les details SQL de pgvector. Cette separation facilite les tests et permettrait de changer le modele d'embedding ou la base vectorielle sans reecrire l'interface.

7 Choix de conception

Plusieurs choix ont ete faits pour concilier exigences academiques, performance locale et clarte de demonstration. Le premier choix est d'utiliser PostgreSQL comme base unique pour les donnees relationnelles et vectorielles. Cette approche simplifie l'exploitation : les filtres temporels, niveaux de severite, templates et vecteurs sont interroges dans le meme systeme.

Le deuxieme choix est de vectoriser les messages normalises plutot que chaque message brut independamment. Sur un dataset de logs structures, les templates representent le sens operationnel des evenements, tandis que les valeurs variables representent souvent des parametres. Cette vectorisation rend les groupes d'erreurs plus stables et accelere fortement l'indexation.

Le troisieme choix est de conserver une recherche par mots-cles. Elle ne remplace pas la recherche semantique, mais elle reste pertinente pour des investigations precises. Par exemple, une adresse IP ou un port doit etre retrouve lexicalement. La comparaison entre les deux methodes montre donc leur complementarite.

Le dernier choix est de fournir une API en plus de l'interface. Une interface Streamlit seule aurait ete plus rapide a ecrire, mais moins propre architecturalement. Avec FastAPI, la demonstration web, les tests et d'eventuels clients externes consomment le meme contrat.

8 Vectorisation

La vectorisation est realisee avec Sentence-Transformers. La commande `tp5-log-search embed` charge le modele, recupere les messages normalises distincts non encore vectorises, calcule les embeddings par lots, puis les insere dans la table `message_embeddings`.

Cette approche est volontaire. Dans OpenSSH, beaucoup de lignes partagent le meme template. Calculer et stocker un embedding duplique pour chaque ligne brute gaspillerait du temps CPU et de l'espace disque sans ajouter beaucoup de signal semantique. En vectorisant les templates normalises, les 638 947 logs restent recherchables par jointure avec une table vectorielle compacte et indexee.

Les vecteurs sont normalises cote modele. La similarite retournee par l'API est :

```
1 - (embedding <=> query_embedding)
```

ou `<=>` est l'operateur de distance cosinus de pgvector.

9 Recherche et analyse

La plateforme expose quatre familles de fonctionnalites.

9.1 Recherche semantique

L'utilisateur formule une requete en langage naturel, par exemple :

```
brute force ssh authentication failure
```

Le systeme vectorise cette requete, interroge pgvector et retourne les logs les plus proches. Cette recherche peut retrouver des messages pertinents meme si les mots exacts ne sont pas presents.

9.2 Recherche par mots-cles

La recherche par mots-cles utilise ILIKE et l'index trigramme. Elle sert de baseline pour comparer les resultats avec la recherche semantique. Cette comparaison est utile pedagogiquement, car elle montre les limites d'une recherche lexicale stricte sur des logs variables.

9.3 Logs similaires

L'endpoint `/logs/{id}/similar` recupere le vecteur d'un log deja indexe, puis cherche ses voisins les plus proches. Ce cas pratique repond directement a la consigne : retrouver tous les logs similaires a une erreur critique donnee.

9.4 Analytique

Les groupes d'erreurs frequentes sont calcules par agregation sur `event_id`, `event_template` et `level`. L'evolution temporelle agrège les logs par heure ou par jour. Lorsqu'une requete semantique est fournie, le systeme identifie d'abord les evenements les plus proches, puis calcule leur evolution sur l'ensemble de la base.

10 Interface de demonstration

L'interface Streamlit est organisee en quatre onglets : recherche, comparaison, logs similaires et analytique. Elle consomme l'API FastAPI et n'accede jamais directement a la base. Cette separation est importante pour garder une architecture claire et testable.

La page d'accueil affiche les indicateurs principaux : nombre total de logs, nombre de logs vectorises, nombre d'evenements et presence de l'index HNSW. Les tableaux de resultats affichent les identifiants de logs, timestamps, niveaux, evenements, similarites et messages bruts. Les analyses recurrences et temporelles utilisent des graphiques Plotly.

11 Reproductibilite

Le projet est pilote par une CLI unique :

```
tp5-log-search prepare-data
tp5-log-search init-db
tp5-log-search load-db
tp5-log-search embed
tp5-log-search index
tp5-log-search stats
```

Pour une demonstration rapide, l'option `-limit` permet de traiter un sous-ensemble :

```
tp5-log-search run-pipeline --limit 10000
```

Pour la livraison finale, la commande sans limite traite les 638 947 logs :

```
tp5-log-search run-pipeline
```

La configuration est centralisee dans `.env`. Les chemins du dataset, l'URL PostgreSQL, le modele d'embedding, l'annee de log et les tailles de batch sont modifiables sans changer le code.

12 API applicative

L'API FastAPI constitue le contrat applicatif entre la base vectorielle et les clients. Elle a ete concue pour exposer des operations simples, chacune correspondant a un besoin du sujet. La table suivante resume les endpoints principaux.

Endpoint	Fonction
GET /health	Verification de disponibilite de l'API.
GET /stats	Statistiques globales : logs, vecteurs, evenements, periode, index.
POST /search/semantic	Recherche par similarite vectorielle a partir d'une requete libre.
POST /search/keyword	Recherche lexicale par mots-cles pour comparaison.
POST /search/compare	Retour simultane des resultats semantiques et lexicaux.
GET /logs/{id}/similar	Voisins semantiques d'un log deja stocke.
GET /analytics/frequent-errors	Groupes d'erreurs les plus frequents.
GET /analytics/timeline	Evolution temporelle par requete, evenement ou niveau.

Le chargement du modele d'embedding est mis en cache cote API. Sans ce cache, chaque requete semantique rechargerait le modele depuis le disque, ce qui augmenterait fortement la latence. Le cache garde un seul objet `EmbeddingService` par processus FastAPI.

Les requetes applicatives acceptent un parametre `top_k`. Celui-ci borne le nombre de resultats retournees afin d'eviter des reponses trop volumineuses. Les filtres par niveau permettent de limiter la recherche a des logs critiques, erreurs, avertissements ou informations.

13 Scenario de demonstration

Une demonstration complete peut etre realisee en quatre temps. D'abord, l'enseignant ou l'utilisateur lance PostgreSQL avec Docker Compose et execute le pipeline limite pour verifier le fonctionnement sur un echantillon. Ensuite, il lance l'API et l'interface Streamlit. L'interface affiche immediatement les statistiques de la base et la presence de l'index.

Le premier cas pratique consiste a chercher une erreur critique. Une requete comme `possible break in ssh server` retourne les logs du template `POSSIBLE BREAK-IN ATTEMPT`. Le resultat prouve que la recherche n'est pas limitee a une correspondance exacte des mots.

Le deuxieme cas pratique porte sur les erreurs frequentes. L'onglet analytique affiche les groupes dominants : echecs de mot de passe, utilisateurs invalides, fermetures de connexion et echecs d'authentification PAM. Ces groupes correspondent aux evenements recenses dans LogHub et donnent une vision synthetique du comportement du service SSH.

Le troisieme cas pratique analyse l'evolution temporelle. A partir d'une requete semantique, l'application identifie les evenements proches puis agrege leurs occurrences par jour ou par heure. Cette vue permet de reperer des periodes de forte activite suspecte, par exemple une concentration d'echecs d'authentification.

Enfin, l'onglet comparaison affiche cote a cote la recherche semantique et la recherche par mots-cles. Cette comparaison illustre l'interet des embeddings : la recherche semantique peut retrouver des messages proches meme lorsque le vocabulaire exact differe, tandis que la recherche par mots-cles reste utile pour des termes precis comme une adresse IP ou un nom d'utilisateur.

14 Evaluation attendue

L'evaluation du projet combine des criteres fonctionnels, techniques et pedagogiques. Sur le plan fonctionnel, le systeme doit charger les donnees, remplir la base, vectoriser les logs et retourner des resultats pertinents depuis l'interface. Sur le plan technique, le pipeline doit etre reproductible, le schema doit contenir une colonne vectorielle indexee, et les commandes doivent etre documentees.

La qualite des resultats se verifie qualitativement. Pour une requete sur les echecs de mot de passe, les premiers resultats doivent appartenir aux templates `Failed password` ou `authentication failure`. Pour une requete sur les tentatives d'intrusion, les resultats doivent faire apparaitre les messages de type `POSSIBLE BREAK-IN ATTEMPT` ou utilisateurs invalides. Pour les analyses frequentes, les evenements les plus representes doivent correspondre aux occurrences indiquees par LogHub.

La performance depend de la machine. Le pretraitement Spark du dataset complet reste raisonnable sur un poste local. La vectorisation par templates distincts reduit fortement le temps de calcul, car elle evite de traiter plusieurs centaines de milliers de messages quasi identiques. La recherche pgvector avec HNSW permet ensuite une latence compatible avec une demonstration interactive.

15 Tests et validation

Les tests unitaires valident le parsing OpenSSH, la normalisation des messages, la classification de niveau, la conversion des vecteurs au format pgvector et le parsing de la CLI. Les tests d'intégration se font avec le pipeline limite, car ils impliquent Spark, PostgreSQL et le modèle d'embedding.

Les critères d'acceptation sont :

- le prétraitement Spark produit les sorties CSV et Parquet ;
- PostgreSQL contient les logs et templates chargés ;
- la table `message_embeddings` couvre les messages normalisés distincts ;
- l'index HNSW existe sur la table `message_embeddings` ;
- une requête sémantique retourne des messages OpenSSH cohérents ;
- la comparaison mots-cles/sémantique retourne deux listes distinctes ;
- l'interface web communique avec l'API locale ;
- le rapport compile en PDF avec LaTeX.

16 Limites et améliorations

Le dataset OpenSSH ne contient pas l'année dans le timestamp. L'année configurée est donc une hypothèse technique pour permettre les analyses temporelles. Une extension possible serait de croiser les logs avec une source externe contenant l'année exacte.

La vectorisation par template est efficace et adaptée au dataset LogHub. Pour un environnement de production avec des messages plus variés, on pourrait combiner deux niveaux de vecteurs : un vecteur de template pour le regroupement et un vecteur de message brut pour la recherche fine.

Enfin, l'index HNSW privilégie la vitesse de recherche. Pour certaines évaluations, une recherche exacte sans index pourrait servir de référence afin de mesurer le compromis entre performance et rappel.

17 Conclusion

Le projet livre une chaîne complète de recherche sémantique et analytique sur logs massifs. Spark assure le traitement batch, PostgreSQL et pgvector fournissent la base vectorielle indexée, Sentence-Transformers apporte la représentation sémantique, FastAPI expose les fonctionnalités et Streamlit fournit la démonstration.

La solution couvre les cas pratiques demandés : retrouver les logs similaires à une erreur critique, identifier les groupes d'erreurs fréquentes, analyser leur évolution temporelle et comparer la recherche sémantique avec une recherche par mots-cles. Elle reste modulaire, reproductible et adaptable à d'autres jeux de logs.